JETBRAINS

Integrating Codex Into IntelliJ

Anna Zhdan
Principal Engineer

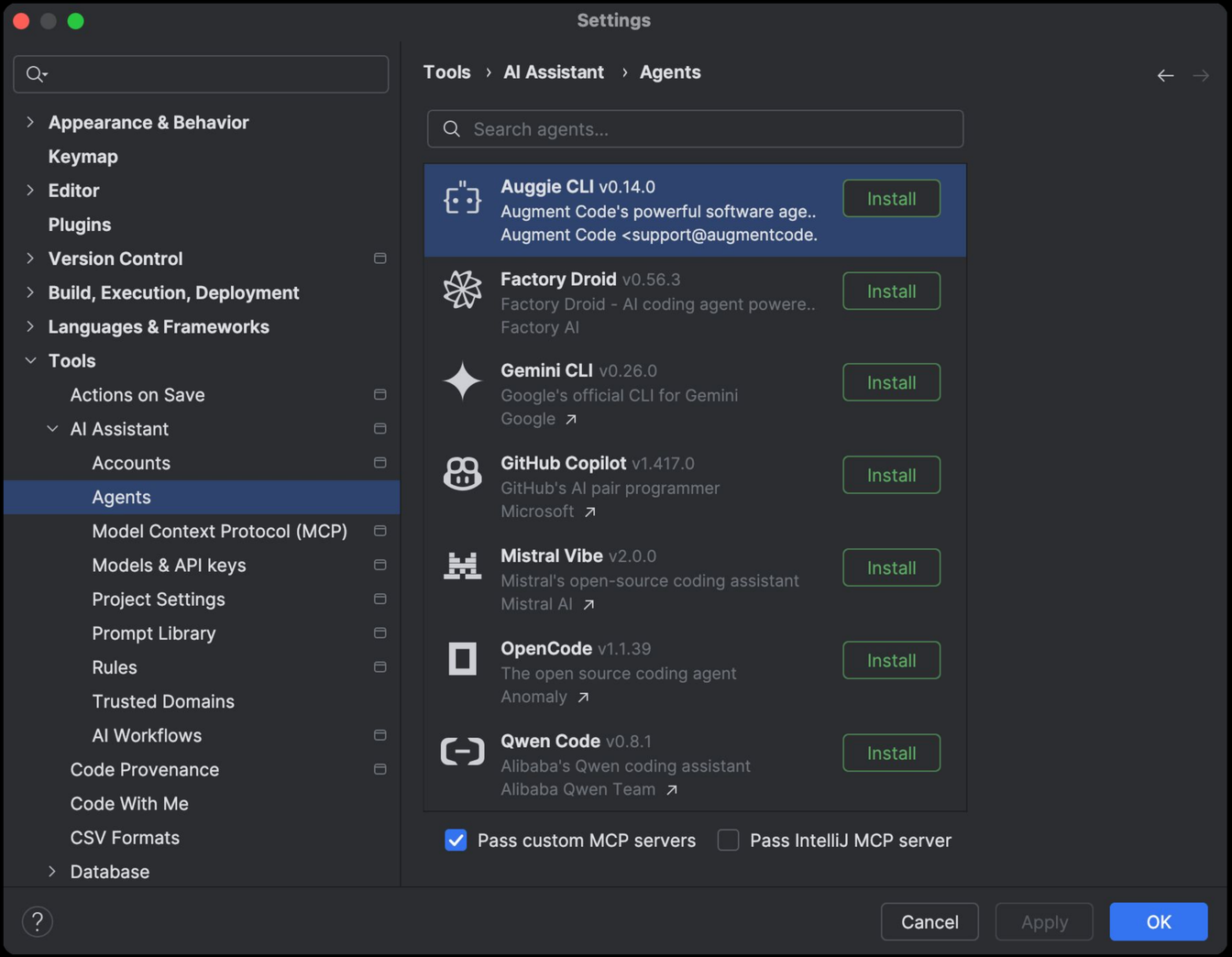
Codex

Building Blocks

Agent/Client
Protocol

Agent Client Protocol

Allows to integrate different coding agents into the IDE



Ways to Integrate Codex

Codex SDK
- CLI-like interface
- Perfect for CI/CD
- Codex as a subagent

Codex App Server
- Events streaming
- Authentication flow
- JSON-RPC 2.0 over stdio

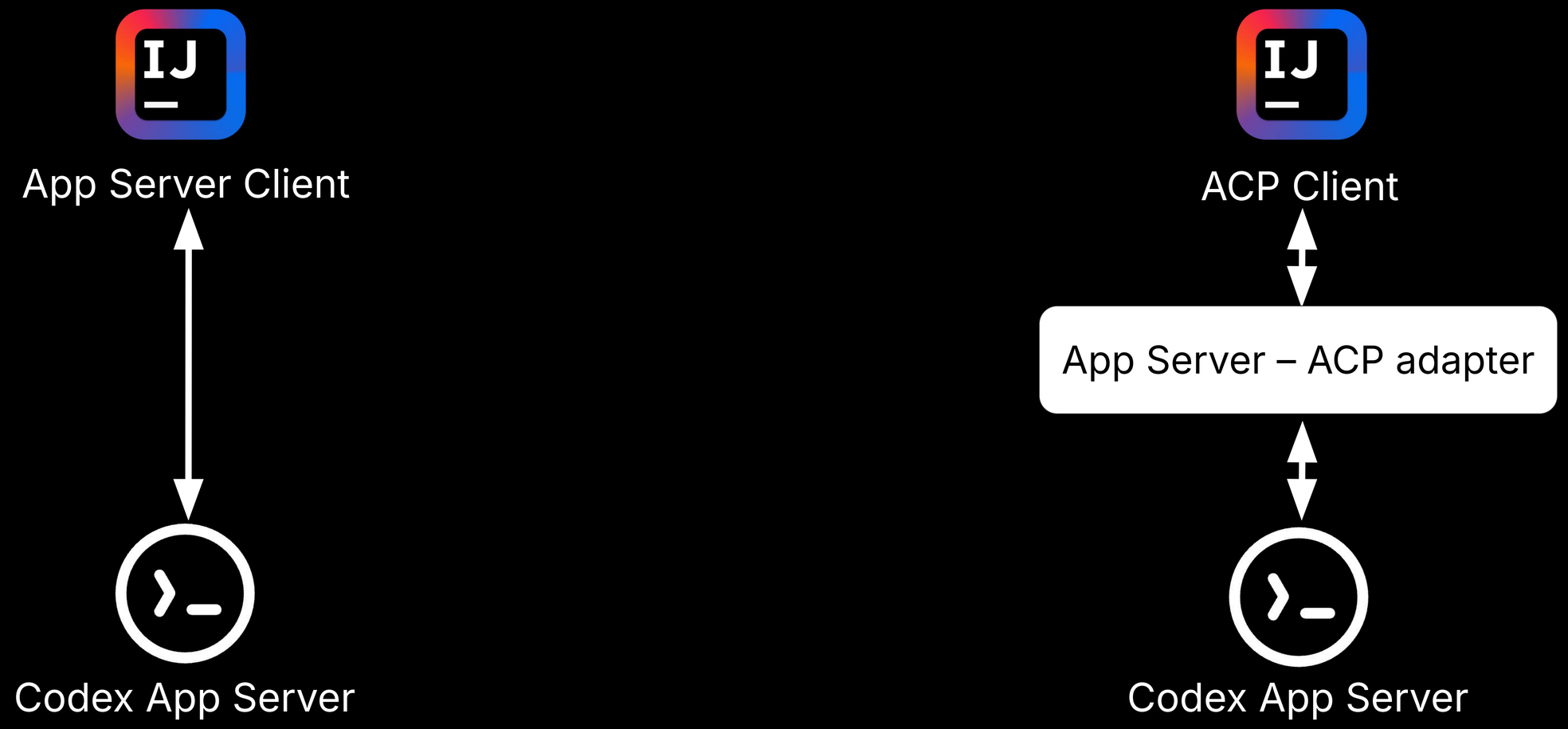
App Server Flow

- Start codex in app server mode via `codex app-server`
- Initialize
- Start or resume a thread
- Begin a prompt turn
- Stream events
- End a prompt turn

App Server vs ACP

App Server	ACP
initialize	initialize
thread/start	session/new
thread/resume	session/load or session/resume
turn/start	session/prompt
item/commandExecution/requestApproval	session/request_permission
item/fileChange/requestApproval	

Native Integration vs App Server Adapter



Native Integration vs App Server Adapter

- More straightforward
- All features are accessible

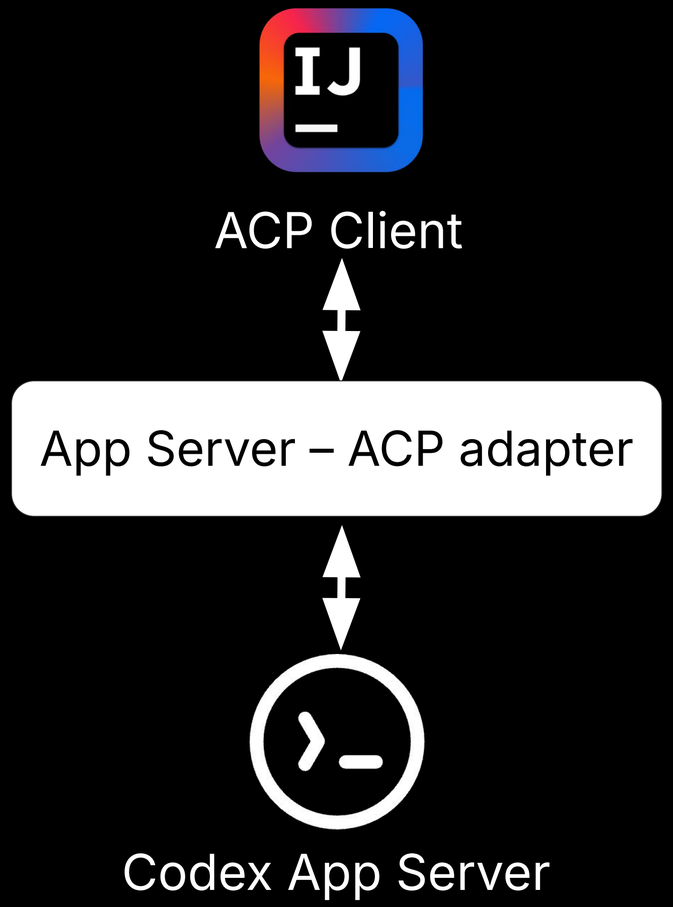
- Custom integration
- Leaking of abstractions

- More dogfooding
- Clear API boundaries
- Easy to test*

- Some features are harder to support
- Maintain a separate adapter

App Server – ACP adapter

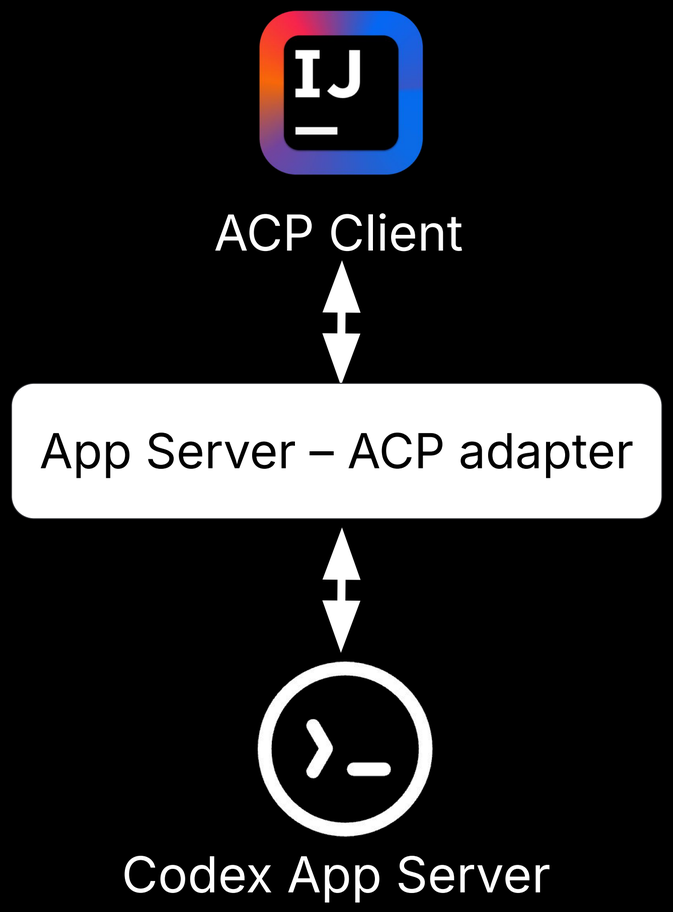
```
// Codex REQUESTS from adapter: { itemId, command, reason }
// Adapter REQUESTS from IDE:   { sessionId, toolCall, options }

async handleCommandExecution(
    params: CommandExecutionRequestApprovalParams
): Promise<CommandExecutionRequestApprovalResponse> {
    // Convert Codex approval request → ACP permission request
    const acpRequest = this.buildCommandPermissionRequest(sessionId, params);

    // Forward to IDE (this blocks until user responds!)
    const response = await this.connection.requestPermission(acpRequest);

    // Convert ACP response → Codex response
    return this.convertCommandResponse(response);
    // "allow_once" → { decision: "accept" }
    // "allow_always" → { decision: "acceptForSession" }
    // "reject_once" → { decision: "decline" }
}
```



How do we know it works?

Why QA for agent integrations is **hard**

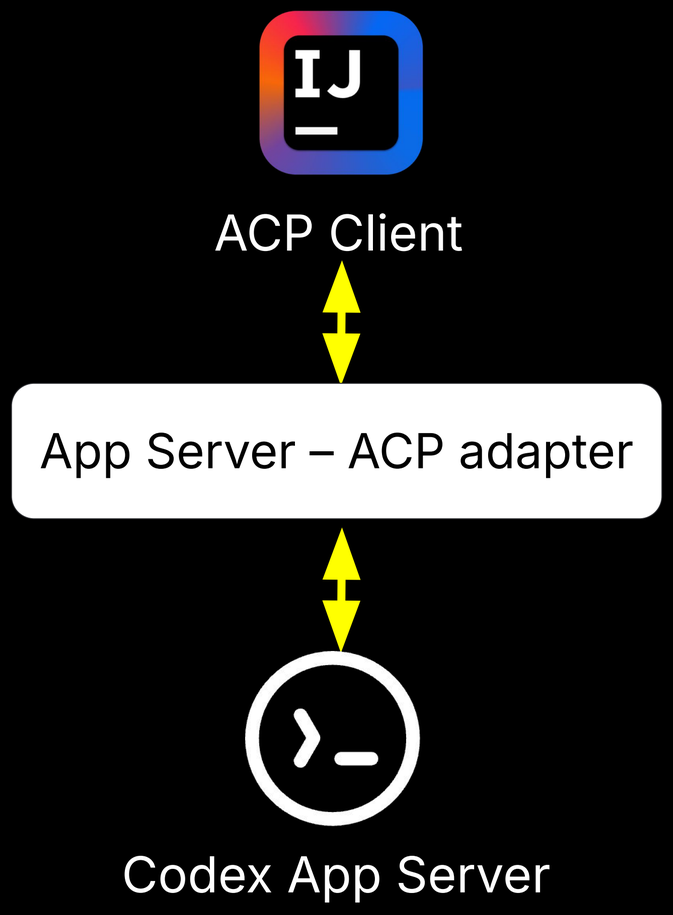
- Testing integrations is hard
- Testing non-deterministic systems is hard
- Testing of rapidly developing systems is hard
- Complex infrastructure for evals
- Running an agent costs money



How do we know it works?

What worked for us

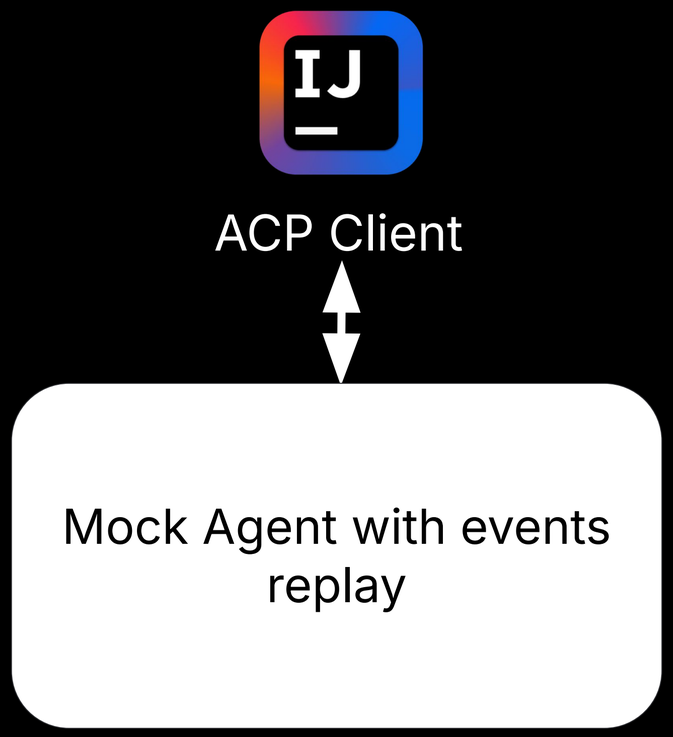
- If something can be tested deterministically – do it
- Separate tests for all the parts
- Record all the JSON RPC traffic, can replay events in tests



How do we know it works?

What worked for us

- If something can be tested deterministically – do it
- Separate tests for all the parts
- Record all the JSON RPC traffic, can replay events in tests
- Mock “agent” that just replays events for reproducing bugs



Thank you!

Anna Zhdan
anna.zhdan@jetbrains.com